Lab Assignment 16.2

Program	**: B. Tech (CSE)**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**: III**
Academic Session	**: 2025-2026**
Name of Student	**: Sruthi**
Enrollment No.	**: 2403A51L10**
Batch No.	**: 51**
Date	**: 17-03-2026**

Task 1:

Prompt: Create a Students, Courses, Enrolments tables and include primary keys, foreign keys and use appropriate data types and ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Code:

```sql
-- Create Students table
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);
-- Create Courses table
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name VARCHAR(100) NOT NULL,
    course_code VARCHAR(10) UNIQUE NOT NULL
);
-- Create Enrollments table
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    enrollment_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Output:

```
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> sqlite3 student.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .read script.sql
sqlite> .tables
Courses      Enrollments  Students
```

Explanation:

- The Students table has columns for student_id, first_name, last_name, and email. The student_id is the primary key.
- The Courses table has columns for course_id, course_name, and course_code. The course_id is the primary key.
- The Enrollments table has columns for enrollment_id, student_id, course_id, and enrollment_date. The enrollment_id is the primary key. The student_id and course_id columns are foreign keys that reference the Students and Courses tables, respectively, to establish relationships between the tables.

Task 2:

Prompt: Insert students, courses, and enrollments records into the respective tables. And Verify data using SELECT statements.

Code:

```sql
-- Insert students
INSERT INTO Students (first_name, last_name, email) VALUES ('John', 'Doe','john.doe@example.com');
INSERT INTO Students (first_name, last_name, email) VALUES ('Jane', 'Smith','jane.smith@example.com');
INSERT INTO Students (first_name, last_name, email) VALUES ('Alice', 'Johnson','alice.johnson@example.com');
-- Insert courses
INSERT INTO Courses (course_name, course_code) VALUES ('Mathematics', 'MATH101');
INSERT INTO Courses (course_name, course_code) VALUES ('Physics', 'PHYS101');
INSERT INTO Courses (course_name, course_code) VALUES ('Chemistry', 'CHEM101');
INSERT INTO Courses (course_name, course_code) VALUES ('Biology', 'BIOL101');
-- Insert enrollments
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (1, 1, '2023-01-15');
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (1, 2, '2023-01-16');
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (2, 1, '2023-01-17');
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (2, 3, '2023-01-18');
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (3, 4, '2023-01-19');
```

Output:

```
sqlite> SELECT * FROM Students;
1|John|Doe|john.doe@example.com
2|Jane|Smith|jane.smith@example.com
3|Alice|Johnson|alice.johnson@example.com
sqlite> SELECT * FROM Courses;
1|Mathematics|MATH101
2|Physics|PHYS101
3|Chemistry|CHEM101
4|Biology|BIOL101
sqlite> SELECT * FROM Enrollments;
1|1|1|2023-01-15
2|1|2|2023-01-16
3|2|1|2023-01-17
4|2|3|2023-01-18
5|3|4|2023-01-19
```

Explanation:

- The INSERT INTO statement is used to add new records to a table.
- For the Students table, we specify the columns (first_name, last_name, email) and provide corresponding values for each student.
- For the Courses table, we specify the columns (course_name, course_code) and provide
- corresponding values for each course.
- For the Enrollments table, we specify the columns (student_id, course_id, enrollment_date) and provide corresponding values for each enrollment, linking students to courses with the enrollment date.
- Verify data using SELECT statements.

Task 3:

Prompt: Generate SQL queries for basic CRUD operations:

• Retrieve all courses

• Update a student's email

• Delete an enrollment record

• Validate the changes using SELECT statements.

Code:

```sql
-- Retrieve all courses
SELECT * FROM Courses;
-- Update a student's email
UPDATE Students SET email = 'new.email@example.com' WHERE student_id = 1;
-- Delete an enrollment record
DELETE FROM Enrollments WHERE enrollment_id = 1;
-- Validate the changes using SELECT statements
SELECT * FROM Students WHERE student_id = 1;
SELECT * FROM Enrollments WHERE enrollment_id = 1;
```

Output:

```
sqlite> SELECT * FROM Courses;
1|Mathematics|MATH101
2|Physics|PHYS101
3|Chemistry|CHEM101
4|Biology|BIOL101
sqlite> UPDATE Students SET email = 'new.email@example.com' WHERE student_id = 1;
sqlite> DELETE FROM Enrollments WHERE enrollment_id = 1;
sqlite> SELECT * FROM Students WHERE student_id = 1;
1|John|Doe|new.email@example.com
sqlite> SELECT * FROM Enrollments WHERE enrollment_id = 1;
```

Explanation:

CRUD operations refer to the four basic functions of persistent storage: Create, Read, Update, and Delete. In the context of the provided SQL script:

- Create: This is demonstrated by the INSERT statements, which add new records to the Students, Courses, and Enrollments tables.
- Read: This is shown by the SELECT statements, which retrieve data from the tables. For example, the query to retrieve all courses and the queries to validate changes after updating a student's email and deleting an enrollment record.
- Update: This is illustrated by the UPDATE statement, which modifies the email of a student in the Students table.
- Delete: This is represented by the DELETE statement, which removes a specific enrollment record from the Enrollments table.

Task 4:

Prompt: Write aggregate queries to count number of students enrolled per course and display courses with more than one student enrolled. Analyse the correctness of results.

Code:

```sql
-- Count number of students enrolled per course
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id;
-- Display courses with more than one student enrolled
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id
HAVING COUNT(student_id) > 1;
```

Output:

```
sqlite> SELECT course_id, COUNT(student_id) AS student_count FROM Enrollments
GROUP BY course_id;
1|1
2|1
3|1
4|1
sqlite> SELECT course_id, COUNT(student_id) AS student_count FROM Enrollments
GROUP BY course_id HAVING COUNT(student_id) > 1;
```

Explanation:

The first query will return the number of students enrolled in each course. The second query will return only those courses that have more than one student enrolled. The correctness of the results can be verified by cross-referencing the enrollment data in the Enrollments table with the course information in the Courses table. If the counts match the expected values based on the inserted data, then the queries are correct. Additionally, checking for any discrepancies or unexpected results can help identify any issues with the data or the queries themselves.

Task 5:

Prompt: Rewrite inefficient sub queries using JOINs and test both queries for correctness.

Sample input Code:

```
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);
```

Code:

```
-- Rewritten Query using JOIN
SELECT s.* FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Output:

```
sqlite> SELECT s.* FROM Students s JOIN Enrollments e ON s.student_id = e.stud
ent_id WHERE e.course_id = 101;
sqlite>
```

Explanation:

The Sample query uses a subquery to find student IDs enrolled in a specific course, which can be inefficient as it may require scanning the Enrollments table multiple times. The rewritten query uses a JOIN to directly connect the Students and Enrollments tables, allowing for a more efficient retrieval of the desired records. Both queries should return the same results, but the JOIN version is typically faster, especially as the size of the tables increases.